

OS Lab8 实验报告

组员：查科言(2312189) 禹相祐(2312900) 董丰瑞(2311973)

分工

- 练习0（合并 Lab2~Lab7 到 Lab8 框架）：禹相祐
- 练习1（sfs_io_nolock() 读文件）：禹相祐
- 练习2（基于文件系统的 exec/load_icode）：董丰瑞
- Challenge 1-2: 查科言
- 报告撰写：三人合作

实验目标与要求对应关系

- 练习0：将 Lab2/3/4/5/6/7 的实现填入 Lab8 中所有 LAB2~LAB7 标记处，并做适当修改。
- 练习1：实现 SFS 文件系统的“读文件”路径（核心函数 sfs_io_nolock()），正确处理非块对齐头部、整块、非块对齐尾部三类读场景。
- 练习2：实现“从文件系统加载并执行用户程序”的机制：execve 打开 SFS 中的 ELF，解析并加载到进程地址空间，构造用户栈 argc/argv，设置 trapframe，使系统进入 sh，并可在 sh 中运行 exit/hello/...。
- Challenge 1: 完成基于“UNIX的PIPE机制”的设计方案。如果要在ucore里加入UNIX的管道（Pipe）机制，至少需要定义哪些数据结构和接口？（接口给出语义即可。数据结构的设计应当给出一个（或多个）具体的C语言struct定义。请在实验报告中给出设计实现“UNIX的PIPE机制”的概要设方案，你的设计应当体现出对可能出现的同步互斥问题的处理。
- Challenge 2: 完成基于“UNIX的软连接和硬连接机制”的设计方案。如果要在ucore里加入UNIX的软连接和硬连接机制，至少需要定义哪些数据结构和接口？接口给出语义即可。数据结构的设计应当给出一个（或多个）具体的C语言struct定义。请在实验报告中给出设计实现“UNIX的软连接和硬连接机制”的概要设方案，你的设计应当体现出对可能出现的同步互斥问题的处理。

练习1：读文件 sfs_io_nolock() 的实现与理解

1. 功能目标

sfs_io_nolock() 在“读文件”模式下要保证：

- 从文件的 `offset` 位置开始读
- 最多读 `*alenp` 字节
- 实际能读多少取决于文件大小（读到文件末尾就停止）
- 把读到的数据按顺序写进 `buf`
- 返回时用 `*alenp` 写回“实际读到的字节数”

文件在磁盘上按块存放，但读请求按字节给出，所以核心工作是：

把 `[offset, offset+len)` 这段**字节区间**拆成“若干块上的片段”，按顺序读出并拼接到 `buf`。

2. 读范围裁剪：先把“实际能读的范围”定死

实现时先做裁剪，其实是在提前回答两个问题：

- 这次读请求是不是本来就不可能读到任何东西？
- 如果可以读，最多读到哪里必须停？

对应代码行为：

- `offset < 0`、`offset >= SFS_MAX_FILE_SIZE` 等直接认为非法
- `endpos = offset + *alenp`，若超过最大限制就裁剪到 `SFS_MAX_FILE_SIZE`
- **读模式**下再裁剪到真实文件大小 `din->size`
 - `offset >= din->size`：从 EOF 或 EOF 后开始读 → 返回 0
 - `endpos > din->size`：只读到 `din->size`

思考点：

这一步如果不做“在开头就裁剪”，后面会出现两类隐患：

1. 读循环按照原始 `endpos` 跑，可能会去读文件根本不存在的块；
2. `offset + len` 发生溢出时，后续 `blkno/nblks` 都会错，属于“越读越离谱”的错误。
因此这里不是“形式化检查”，而是为了把后续逻辑约束在一个可靠的区间里，让后面的三段式拆分能成立。

3. 三段式读取：为什么不直接一个循环搞定？

实现采用三段式：

1. **首块非对齐部分**：`offset` 在块中间 → 先读到块尾
2. **中间整块部分**：已经对齐 → 能整块读就整块读
3. **尾块非对齐部分**：最后剩一点 → 读块头的部分字节

思考点：

完全可以写成“每次算当前块能读多少，再循环推进”的统一写法，但三段式的优势是：

- 把最容易出错的“非对齐处理”限制在首段和尾段两处
- 中间段变成非常规整的“整块循环”，不夹杂偏移计算，出错概率低
- 对读性能也更友好：中间整块部分走 `rblock`，不会反复做块内偏移读

换句话说，三段式是为了让逻辑更“分区”，每一段只解决一种形态的问题。

4. 关键坐标：文件块号、块内偏移，以及它们在实现中的作用

```
uint32_t blkno = offset / SFS_BLKSIZE;
blkoff = offset % SFS_BLKSIZE;
char *data = (char *)buf;
```

- `blkno`：当前字节位置落在文件的第几个数据块
- `blkoff`：在该块中从第几个字节开始读
- `data`：写入 `buf` 的当前位置（每读一段就向后推进）

同时维护 `alen` 作为累计值。

思考点：

这里刻意让 `offset` 和 `data` 同步推进，等价于保持一个不变式：

“`data` 指向 `buf` 中下一个待写位置；`offset` 指向文件中下一个待读位置；两者之间的推进量始终等于 `alen`。”

这样可以避免“读是对的但写到 `buf` 的位置错位”的问题。

5. 每次真正读之前，必须先做块映射：为什么这一步不能省？

```
sfs_bmap_load_nolock(sfs, sin, blkno, &ino);
```

它把“文件第 `blkno` 块”映射为“磁盘真实块号 `ino`”。

思考点：

如果把文件看成一条连续字节流，很容易误以为 `blkno` 就是磁盘块号。

但实际上 `inode` 里维护了块映射（可能间接块、直接块等），`blkno` 只是文件内部的逻辑序号。

因此每次读块都必须先 `bmap`，否则读到的是“某个磁盘块”，但不一定是这个文件的数据块。

6. 首块非对齐：读到块尾（或读到 `endpos`）

```
if (blkoff != 0) {
    size = (nbks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);

    sfs_bmap_load_nolock(..., blkno, &ino);
    sfs_buf_op(..., data, size, ino, blkoff);

    data += size; offset += size; alen += size; blkno++;
}
```

- `size` 的两种情况：
 - 请求跨越多个块：首段读到块尾 `SFS_BLKSIZE - blkoff`
 - 请求只在一个块内：首段直接读完 `endpos - offset`

思考点：

这里的关键不是“把首段读了”，而是“读完首段后 `offset` 对齐到块边界”。

因为只要 `offset` 对齐，后面就可以用完全规整的整块读取逻辑，不再夹杂块内偏移。

所以首段的设计本质上是在为中段创造条件。

7. 中间整块：尽可能用整块读，减少复杂度

```
while (offset + SFS_BLKSIZE <= endpos) {
    sfs_bmap_load_nolock(..., blkno, &ino);
    sfs_block_op(..., data, ino, 1);

    data += SFS_BLKSIZE; offset += SFS_BLKSIZE; alen += SFS_BLKSIZE; blkno++;
}
```

- 循环条件保证本轮能读满一个块
- 每轮只做：映射 → 整块读 → 推进

思考点：

整块读的价值不仅是效率，也是正确性：

当一段逻辑只处理“整块”时，就没有块内偏移、没有“读多少合适”的决策，错误空间显著变小。

因此中段尽量走整块，是一种“用结构化约束降低 bug 风险”的写法。

8. 尾块非对齐：剩多少读多少（从块头开始）

```
if (offset < endpos) {
    size = endpos - offset;

    sfs_bmap_load_nolock(..., blkno, &ino);
    sfs_buf_op(..., data, size, ino, 0);

    alen += size;
}
```

- 尾段读的是当前块的前 `size` 字节
- 块内偏移传 `0`，因为此时 `offset` 已经对齐

思考点：

尾段能用 `offset=0` 的前提是：

首段如果存在就把 `offset` 补齐了；中段每次推进都是整块。

这等价于把“对齐性”当成了三段式之间的契约：

- 首段负责把 `offset` 拉到对齐
- 中段保证一直对齐推进
- 尾段因此可以简化成从块头读

9. `alen` 回写 `*alenp`：为什么不直接用 `(endpos - startpos)`？

最后回写：

```
*alenp = alen;
```

思考点：

理论上如果所有读都成功且无错误，`alen` 会等于 `endpos - startpos`。

但实际实现里仍然需要 `alen`，原因是：

- 任何一次 `bmap` 或 `rbuf/rblock` 失败，都可能提前跳转 `out`
- 若提前退出，需要准确告诉上层“已经成功读了多少字节”，而不是“本来计划读多少”
- 这样上层可以按“部分成功”语义处理（尤其是在更复杂的读写路径里）

因此 `alen` 是“实际完成量”，不是“计划量”。

练习2：基于文件系统加载并执行 `load_icode(fd, argc, kargv)` 的实现与理解（更具体 + 思考过程）

1. 功能目标（从“能运行起来”倒推需要满足的条件）

`load_icode(fd, argc, kargv)` 最终要达到的效果不是“把文件读进来”，而是让一次 `sret` 之后立刻满足以下事实：

- **取指能成功**： `PC = elf.e_entry` 对应的用户虚拟地址在新页表里有映射，且权限允许取指
- **访问数据能成功**：ELF 段中的数据页存在，读写权限符合段属性
- **栈能用**： `SP` 指向的用户栈页存在且可读写
- **参数可用**：用户态入口拿到的 `argc/argv` 在用户栈上，且 `argv[i]` 指向的字符串也在用户空间有效地址里
- **切换确实用了新地址空间**：CPU 的页表寄存器已经指向新 `pgdir`，TLB 不会还缓存旧映射

思考过程：

实现顺序基本可以按“风险最高的先固定”的思路来排：

1. 先确认文件是 ELF（否则后面解析全是错的）；
2. 先造一套新的 `mm + pgdir`（否则无法给新程序准备干净空间）；
3. 再按 ELF 描述把段搬进来（保证取指、数据访问）；
4. 再建立栈并把参数放进去（保证程序一进来就能读 `argv`）；
5. 最后切页表 + 填 `trapframe`（把控制权交给新程序）。

2. ELF 头校验：先确认“后续结构体读取”是可靠的

```
load_icode_read(fd, &elf, sizeof(elf), 0);
if (elf.e_magic != ELF_MAGIC) ...
```

- `load_icode_read` 通过 `fd` 从文件系统读取，意味着**数据来源是磁盘文件**而不是内核里某段固定内存
- `ELF_MAGIC` 不对就必须停止：因为后面会用 `elf.e_phoff / e_phnum / e_entry` 等字段去定位 Program Header，若结构不可信，最直接的后果是读到错误偏移甚至越界

思考过程：

这一步的意义在于“把不确定性尽早排除”。如果 ELF 头错了还继续：

- 可能把随机数据当成段信息
- 随后 `mm_map` 会映射奇怪的地址范围
- 更糟的是 `pgdir_alloc_page` 会在用户空间乱分配页导致系统状态被污染
因此这里属于“宁可早失败，也不能把错误带入内存管理阶段”。

3. 新地址空间建立: `mm_create` + `setup_pgdir` (先搭骨架)

```
mm = mm_create();
setup_pgdir(mm);
```

- `mm_create()`: 创建一套新的内存管理结构, 后续 VMA (虚拟内存区域) 都挂在它上面
- `setup_pgdir(mm)`: 创建并初始化页表根 (`mm->pgdir`), 并把必要的内核映射拷贝/建立好 (ucore 通常要求内核部分映射保持可用)

思考过程:

如果直接在旧 `mm` 上改, 会出现两个问题:

1. 旧进程的映射与新程序冲突 (尤其是用户空间地址布局不同)
2. 一旦加载中途失败, 很难恢复原地址空间
因此更稳的策略是“在新 `mm` 上完整构建, 全部成功后一次性切换”。

4. 找到 `gp`: 为了让 RISC-V 小数据访问不出错 (实现里做的额外可靠性)

实现中额外扫描 section header + symbol table 来定位 `__global_pointer$`:

- 如果 ELF 里存在符号表, 就找名字为 `"__global_pointer$"` 的符号
- 找到后把 `gp = st_value` 记录下来, 后面写入 `tf->gpr.gp`

思考过程:

这一步并不是所有实验版本都要求, 但在 RISC-V 上很关键:

很多编译器会生成基于 `gp` 的小数据访问, 如果 `gp` 不正确, 程序会在访问全局/静态数据时出现异常 (表现为“段明明加载对了但运行还是崩”)。

因此这里属于“保证能跑稳”的补强, 而不是为了让代码看起来复杂。

5. 按 Program Header 逐段加载: 让用户空间内容与 ELF 描述一致

核心循环:

1. 读第 `i` 个 `proghdr`
2. 只处理 `ELF_PT_LOAD`
3. 检查 `p_filesz <= p_memsz`
4. 检查段地址在用户可访问范围
5. 建 VMA + 分配页 + 拷贝文件数据 + BSS 清零

5.1 读取并筛选可加载段

```
load_icode_read(fd, &ph, sizeof(ph), elf.e_phoff + i * sizeof(struct proghdr));
if (ph.p_type != ELF_PT_LOAD) continue;
```

- ELF 里可能还有解释器信息、动态链接信息等段, 实验只关心 `LOAD` 段

思考过程：

如果把非 `LOAD` 段也当作需要映射，会导致：

- 映射无意义地址区间
 - 权限设置混乱
 - 对用户空间布局造成污染
- 因此只处理 `LOAD` 是最清晰的策略。

5.2 合法性检查：避免“文件描述要求本身就不成立”

```
if (ph.p_filesz > ph.p_memsz) return -E_INVAL;
if (!USER_ACCESS(ph.p_va, ph.p_va + ph.p_memsz)) return -E_INVAL;
```

- `p_filesz > p_memsz`：表示“文件里要装入的内容比内存段还大”，明显矛盾
- `USER_ACCESS`：强制段必须落在允许的用户虚拟地址范围，避免把段映射到内核区

思考过程：

这些检查属于“防御式”：即便测试样例不会给坏 ELF，也要保证实现不会被异常输入拖垮内核。

5.3 权限与 VMA：让“段属性”反映到页表权限

```
uint32_t vm_flags = 0, perm = PTE_U;
if (ph.p_flags & ELF_PF_X) { vm_flags |= VM_EXEC; perm |= PTE_X | PTE_R; }
if (ph.p_flags & ELF_PF_W) { vm_flags |= VM_WRITE; perm |= PTE_W | PTE_R; }
if (ph.p_flags & ELF_PF_R) { vm_flags |= VM_READ; perm |= PTE_R; }

mm_map(mm, ph.p_va, ph.p_memsz, vm_flags, NULL);
```

- `mm_map` 先登记 VMA（逻辑层面：这段地址范围属于进程）
- `perm` 决定页表项权限（物理映射层面：该页是否可读/写/执行）

思考过程：

这里故意同时维护 `vm_flags` 和 `perm`，因为两者职责不同：

- VMA 是“区域属性记录”，后续如缺页处理会查 VMA 决定是否合法
 - PTE 权限是硬件最终执行的限制
- 即便实验里缺页机制不完整，仍然应该保持这两层一致，这样设计不会和后续实验/扩展冲突。

5.4 逐页分配并填充：文件内容 + BSS 语义一次完成

```
uintptr_t file_end = ph.p_va + ph.p_filesz, mem_end = ph.p_va + ph.p_memsz;

for (la = ph.p_va; la < mem_end; la += PGSIZE) {
    page = pgdir_alloc_page(mm->pgdir, la, perm);
    kva = page2kva(page);
    page_off = PGOFF(la);
    memset(kva, 0, PGSIZE);

    if (la < file_end) {
        off = la - ph.p_va;
        size = file_end - la;
        copy = PGSIZE - page_off;
```

```

        if (size < copy) copy = size;
        load_icode_read(fd, kva + page_off, copy, ph.p_offset + off);
    }
}

```

这里有几个“容易写错但实现里处理了”的点：

- **先清零整页**：保证 BSS 部分天然为 0
这样不需要单独再开一个循环处理 BSS
- **page_off 的意义**：当 `ph.p_va` 不是页对齐时，首页需要从页内偏移处开始填数据
- **copy 的计算**：一次最多填满当前页剩余空间 `PGSIZE - page_off`
但如果文件剩余不足这一页，就取更小值
- **文件偏移** `ph.p_offset + off`：`off = la - ph.p_va` 保证虚拟地址推进多少，文件读取位置也推进多少

思考过程：

这段实现的目标是把“段描述”变成“内存现实”。

最核心的正确性条件是：

- 用户虚拟地址 `[ph.p_va, ph.p_va+p_filesz)` 的内容 == 文件中 `[ph.p_offset, ph.p_offset+p_filesz)`
- 用户虚拟地址 `[ph.p_va+p_filesz, ph.p_va+p_memsz)` 全为 0

把“清零”放在每页一开始做，比单独处理 BSS 更稳，因为它天然覆盖了跨页、非对齐等情况。

6. 用户栈建立与参数放置：确保入口代码能正确读取 argv

6.1 先把栈区间变成“有效可访问的用户内存”

```

mm_map(mm, USTACKTOP - USTACKSIZE, USTACKSIZE, VM_READ | VM_WRITE | VM_STACK,
NULL);

for (la = USTACKTOP - USTACKSIZE; la < USTACKTOP; la += PGSIZE) {
    page = pgdir_alloc_page(mm->pgdir, la, PTE_U | PTE_R | PTE_W);
    memset(page2kva(page), 0, PGSIZE);
}

```

思考过程：

这里先 `mm_map` 再逐页分配，是为了保证：

- VMA 层面认为这是合法栈区域
- 页表层面确实有物理页
如果只做其中之一，要么“区域合法但没有页”，要么“有页但缺少 VMA 记录”，都会在后续扩展里埋雷。

6.2 参数入栈：先放字符串，再放 argv 指针数组（顺序不能反）

字符串阶段：

- 从 `sp = USTACKTOP` 往下压
- 每压一个字符串，记录它在用户空间中的地址到 `uargv[i]`

```
for (int i = argc - 1; i >= 0; i--) {
    len = strlen(kargv[i]) + 1;
    sp -= len;
    page = get_page(mm->pgdir, sp, NULL);
    memcpy(page2kva(page) + PGOFF(sp), kargv[i], len);
    uargv[i] = sp;
}
```

argv 数组阶段：

- 先对齐 `sp`
- 预留 `(argc+1)` 个指针位置（最后一个为 NULL）
- 把 `uargv[i]` 写入这些位置

```
sp = ROUNDDOWN(sp, sizeof(uintptr_t));
sp &= ~((uintptr_t)0xF);
sp -= (argc + 1) * sizeof(uintptr_t);
argv_user = sp;
...
```

思考过程：

“先字符串后 argv”的原因是：

argv 数组里存的是字符串地址。如果先写 argv，再压字符串，字符串的压栈会改变 `sp`，从而使之前写入的 argv 指针失效。

因此实现选择了更稳定的方式：先把最终字符串位置固定下来，再把这些地址写进 argv 数组。

6.3 对齐与留空：不是装饰，而是避免启动阶段的隐性错误

```
sp &= ~((uintptr_t)0xF); // 16-byte aligned
...
sp -= 0x80;
sp &= ~((uintptr_t)0xF);
```

- 16 字节对齐：满足常见 ABI 约定，避免某些用户代码假设栈对齐后执行对齐访问时出错
- 额外留 `0x80`：避免入口代码早期使用栈（保存寄存器、构造栈帧）时紧贴栈顶边界造成不稳定

7. 启用新地址空间：必须切换页表并刷新 TLB

```
current->mm = mm;
current->pgdir = PADDR(mm->pgdir);
lsatp(current->pgdir);
flush_tlb();
```

思考过程：

前面做的映射都写在 `mm->pgdir` 里，如果不切换：

- `tf->epc` 即便指向 ELF 入口，CPU 仍用旧页表翻译，取指失败
 - `argv_user` 即便写进了栈，也可能在旧页表下不可访问
- 因此这里是“让前面所有工作真正生效”的开关。

8. 设置 trapframe：把“将要发生的用户态启动现场”描述清楚

```
tf->gpr.sp = sp;
tf->gpr.a0 = argc;
tf->gpr.a1 = argv_user;
if (gp != 0) tf->gpr.gp = gp;

tf->status = (read_csr(sstatus) | SSTATUS_SPIE) & ~SSTATUS_SPP;
tf->epc = elf.e_entry;
```

- `sp`：指向构造好的用户栈
- `a0/a1`：按调用约定传参（入口代码通常会从寄存器读取 `argc/argv`）
- `gp`：若能从 ELF 解析到 `__global_pointer$`，就填进去，确保小数据访问正确
- `status`：清掉 SPP（确保 `sret` 回到用户态），同时让返回后中断可用
- `epc`：用户态第一条指令地址

思考过程：

trapframe 相当于“返回用户态时 CPU 要恢复的寄存器快照”。
因此这一步不是“设置几个字段”，而是在保证：

- `PC` 指向可取指的位置
 - `SP` 指向可读写的栈页
 - 参数寄存器指向有效的用户地址
- 只要这三类条件成立，新程序启动阶段就能稳定运行。

9. 失败路径清理：避免半构建状态残留

```
bad_cleanup_mmap:
    exit_mmap(mm);
    put_pgdir(mm);
bad_mm:
    mm_destroy(mm);
fail_close_fd:
    sysfile_close(fd);
    return ret;
```

思考过程：

加载过程涉及页表、VMA、物理页分配、文件读取。一旦中途失败，如果不清理：

- 会泄漏物理页
 - 会残留页表页
 - 会使当前进程处于“mm 部分更新”的危险状态
- 因此实现里把清理路径按构建顺序反向释放，保证失败不会污染系统后续运行。

练习1和练习2实验结果

终端输入 `make grade` 后，输出如下：

```

gmake[1]: Entering directory '/home/doufuru/OS/NKU_2025_Autumn_Operating_System_Group15/lab8' + cc tools/mksfs.c + cc user/badang.c +
cc user/libs/dir.c + cc user/libs/file.c + cc user/libs/initcode.S + cc user/libs/panic.c + cc user/libs/stdio.c + cc user/libs/sysc
all.c + cc user/libs/ulib.c + cc user/libs/umain.c + cc libs/hash.c + cc libs/printfmt.c + cc libs/rand.c + cc libs/string.c + cc use
r/badsegment.c + cc user/divzero.c + cc user/exit.c + cc user/faultread.c + cc user/faultreadkernel.c + cc user/forktest.c + cc user/
forktree.c + cc user/hello.c + cc user/matrix.c + cc user/pgdir.c + cc user/priority.c + cc user/sh.c + cc user/sleep.c + cc user/sle
epkill.c + cc user/softint.c + cc user/spin.c + cc user/testbss.c + cc user/waitkill.c + cc user/yield.c create bin/sfs.img (disk0) s
uccessfully. + cc kern/init/entry.S + cc kern/init/init.c + cc kern/libs/readline.c + cc kern/libs/stdio.c + cc kern/libs/string.c +
cc kern/debug/kdebug.c + cc kern/debug/kmonitor.c + cc kern/debug/panic.c + cc kern/driver/clock.c + cc kern/driver/console.c + cc ke
rn/driver/dtb.c + cc kern/driver/ide.c + cc kern/driver/intr.c + cc kern/driver/picirq.c + cc kern/driver/ramdisk.c + cc kern/trap/tr
ap.c + cc kern/trap/trapentry.S + cc kern/mm/default_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc kern/mm/vmm.c + cc kern/syn
c/check_sync.c + cc kern/sync/monitor.c + cc kern/sync/sem.c + cc kern/sync/wait.c + cc kern/fs/file.c + cc kern/fs/fs.c + cc kern/fs
/iobuf.c + cc kern/fs/sysfile.c + cc kern/process/entry.S + cc kern/process/proc.c + cc kern/process/switch.S + cc kern/schedule/defa
ult_sched.c + cc kern/schedule/default_sched_stride.c + cc kern/schedule/sched.c + cc kern/syscall/syscall.c + cc kern/fs/swap/swapfs
.c + cc kern/fs/vfs/inode.c + cc kern/fs/vfs/vfs.c + cc kern/fs/vfs/vfsdev.c + cc kern/fs/vfs/vfsfile.c + cc kern/fs/vfs/vfslookup.c
+ cc kern/fs/vfs/vfspath.c + cc kern/fs/devs/dev.c + cc kern/fs/devs/dev_disk0.c + cc kern/fs/devs/dev_stdin.c + cc kern/fs/devs/dev_
stdout.c + cc kern/fs/sfs/bitmap.c + cc kern/fs/sfs/sfs.c + cc kern/fs/sfs/sfs_fs.c + cc kern/fs/sfs/sfs_inode.c + cc kern/fs/sfs/sfs
_io.c + cc kern/fs/sfs/sfs_lock.c + ld bin/kernel riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img gmake[1]
: Leaving directory '/home/doufuru/OS/NKU_2025_Autumn_Operating_System_Group15/lab8'
-sh execve: OK
-user sh : OK
Total Score: 100/100
doufuru@LAPTOP-0958B7UG:~/OS/NKU_2025_Autumn_Operating_System_Group15/lab8$

```

输出 100/100 !

终端输入 `make qemu` 后, 结果如下:

```

yuxiangyou@MatthewLaptop:~/OS/Final_Lab8$ make qemu
Size: 0x0000000080000000 (128 MB)
End: 0x0000000087ffffff
DTB init completed
memory management: default_pmm_manager
physcial memory map:
memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
Page table directory switch succeeded!
Kernel stack guardians set succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
sched class: RR_scheduler
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (107/10/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
$

```

成功进入 `sh` 用户界面! 再输入一个 `hello` 运行试试:

```
memory management: default_pmm_manager
physcial memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
Page table directory switch succeeded!
Kernel stack guardians set succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
sched class: RR_scheduler
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (107/10/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
Hello world!!.
I am process 3.
hello pass.
```

输出 `hello pass`！证明实现没有问题！

Challenge 1: UNIX Pipe 机制设计方案

1. Pipe 要做到什么（从“使用者”角度）

Pipe 可以理解成：内核里的一条“内存中的水管”，写端把字节倒进去，读端把字节取出来。

需要满足这些最基本的规则：

- `pipe(int fd[2])` 返回两个 fd
 - `fd[0]`：读端
 - `fd[1]`：写端
- 写端 `write(fd[1], data, n)`：把 `n` 个字节按顺序放进管道
- 读端 `read(fd[0], buf, n)`：从管道按顺序取出最多 `n` 个字节
- 如果管道里**没有数据**：
 - 还有写端开着：读端应该“等一等”（阻塞），直到有人写入
 - 所有写端都关了：读端读到结尾，`read` 返回 0（表示 EOF）
- 如果管道里**没有空间**（装满了）：
 - 还有读端开着：写端应该“等一等”（阻塞），直到有人读走数据
 - 所有读端都关了：写端写入应该失败（UNIX 会 SIGPIPE；在 ucore 可先返回一个错误码）

做成这样，shell 才能用 `cmd1 | cmd2` 这种写法把前一个程序的输出接到后一个程序的输入。

2. 设计思路

Pipe 的数据不需要保存到磁盘，它只是运行时传输数据：

- 不需要“文件名”
- 不需要目录项
- 不需要磁盘块分配
- 只需要：一块内存当缓冲区 + 两个端点共享同一份缓冲区

因此建议把 Pipe 做成一种“特殊文件对象”：它仍然通过 fd 使用，但底层不是 SFS inode，而是一段内存结构。

3. 需要哪些数据结构（重点：能读写、能等待、能判断对端是否还在）

3.1 管道本体：环形缓冲区 + 读写位置 + 端点计数 + 等待队列

“环形缓冲区”用来存放字节流：写指针往前走，走到末尾绕回开头；读指针同理。

关键是要能清楚判断“空/满”，避免读写指针绕一圈后分不清状态，所以加入 `used` 表示当前用了多少字节。

```
#define PIPE_BUF_SIZE 4096    // 管道容量（可按需调整）

typedef struct wait_queue wait_queue_t;
typedef struct spinlock spinlock_t;

struct pipe {
    // 内存缓冲区：保存字节流
    uint8_t buf[PIPE_BUF_SIZE];

    // 环形指针
    uint32_t rpos;    // 下次从哪里读
    uint32_t wpos;    // 下次从哪里写
    uint32_t used;    // 当前缓冲区里有多少字节（0..PIPE_BUF_SIZE）

    // 还有多少“读端/写端”活着，用来决定 EOF / 写失败
    uint32_t nreaders;
    uint32_t nwriters;

    // 保护上面这些共享字段，避免同时读写把指针弄乱
    spinlock_t lock;

    // 等待队列：
    // 读端等数据（used==0 且还有写端）
    wait_queue_t rwait;
    // 写端等空间（used==PIPE_BUF_SIZE 且还有读端）
    wait_queue_t wwait;
};
```

思考点（为什么要有 `nreaders/nwriters`）：

- 只知道“现在空”不够，还要知道“以后会不会有人写”。
如果没人会写，读就不该一直等，应当返回 0 (EOF)。
- 只知道“现在满”不够，还要知道“以后会不会有人读”。
如果没人会读，写也不该一直等，应当立刻失败。

3.2 与 fd 的连接：让 pipe 看起来“像文件”

ucore 里通常有 `struct file` 表示一个 fd 背后的对象。pipe 可以复用同一套接口：read/write/close，只是实现不同。

```
struct pipe_file {
    struct pipe *p;
    int readable;    // 1 表示这是读端
    int writable;    // 1 表示这是写端
};
fd[0] 对应的 file: `readable=1,writable=0`
fd[1] 对应的 file: `readable=0,writable=1`
```

4. 至少需要哪些接口（系统调用 + 内核内部入口）

4.1 系统调用

- `int sys_pipe(int fd[2]);`
创建管道并返回两个 fd。
- `int sys_pipe2(int fd[2], int flags);`
支持非阻塞等选项（不是必需项，可先不做）

4.2 file 层操作

- `ssize_t pipe_read(struct file *f, void *buf, size_t len);`
- `ssize_t pipe_write(struct file *f, const void *buf, size_t len);`
- `int pipe_close(struct file *f);`

5. 读写逻辑怎么写

5.1 读：pipe_read 的完整语义

读端要处理三种情况：

1. **有数据**：直接读走
2. **没数据但还有写端**：必须等待
3. **没数据且没有写端**：返回 0 (EOF)

伪代码（强调“睡眠前放锁、醒来再拿锁”这件事）：

```
pipe_read(f, buf, len):
    p = f->pipe

    lock(p->lock)
```

```

while (p->used == 0) {
    if (p->nwriters == 0) {
        unlock(p->lock)
        return 0;                // EOF
    }

    // 这里必须“边睡边放锁”，否则写端进不来写数据，会死锁
    sleep_on(&p->rwait, &p->lock);
    // sleep_on 返回时，锁已经重新拿回来了
}

n = min(len, p->used)
从环形缓冲区拷贝 n 字节到 buf
更新 rpos / used

wakeup(&p->wwait)                // 读走数据后释放空间，叫醒可能在等空间的写端
unlock(p->lock)
return n

```

关键设计理由：

- 读端阻塞时必须释放锁，否则写端无法进入写逻辑，永远唤醒不了读端
- 读走后要唤醒写端，否则写端可能一直在等“有空间”

5.2 写：pipe_write 的完整语义

写端要处理三种情况：

1. **没有读端**：立刻失败（没人接收）
2. **有空间**：写进去
3. **没空间但还有读端**：必须等待

伪代码：

```

pipe_write(f, buf, len):
    p = f->pipe

    lock(p->lock)

    if (p->nreaders == 0) {
        unlock(p->lock)
        return -E_PIPE;        // 写失败: broken pipe
    }

    total = 0
    while (total < len) {

        while (p->used == PIPE_BUF_SIZE) {
            if (p->nreaders == 0) {
                unlock(p->lock)
                return (total > 0) ? total : -E_PIPE;
            }
            sleep_on(&p->wwait, &p->lock);
        }
    }

```

```

    can = min(len-total, PIPE_BUF_SIZE - p->used)
    从 buf+total 拷贝 can 字节到环形缓冲区
    更新 wpos / used
    total += can

    wakeup(&p->rwait)           // 写入后有数据，叫醒可能在等数据的读端
}

unlock(p->lock)
return total

```

关键设计理由：

- 如果读端全关了，写端不应继续等待，否则会出现“写永远卡住”的行为
- 写入后必须唤醒读端，否则读端可能一直在等数据

6. close 逻辑怎么设计

关闭某一端时，要更新计数并唤醒对方，否则对方可能一直睡着：

- 关闭读端：
 - `nreaders--`
 - 唤醒写端等待队列（让写端发现“没人读”并返回错误）
- 关闭写端：
 - `nwriters--`
 - 唤醒读端等待队列（让读端发现 EOF 并返回 0）
- 当 `nreaders==0 && nwriters==0`：释放 `struct pipe`

7. `cmd1 | cmd2` 为什么能跑

用户态 shell 做的事情本质是：

- 建管道 `pipe(fd)`
- 两次 `fork`
- 把一个进程的标准输出改到 `fd[1]`，另一个进程的标准输入改到 `fd[0]`
- 各自 `exec` 成真正的程序

内核只要保证 pipe 的阻塞/EOF/写失败语义正确，就能支持这种组合。

Challenge 2: UNIX 软链接与硬链接机制设计方案

1. 先把两者差别说清楚

可以把文件系统看成两层：

- **目录里的名字 (name)**：例如 `"a.txt"`
- **文件本体 (inode)**：真正记录文件大小、数据块位置等信息

1.1 硬链接 (hard link)

硬链接做的事：增加一个新的名字，指向同一个文件本体（同一个 inode）。

结果：

- 两个名字等价，删掉其中一个名字，另一个还能用
- 只有当所有名字都删掉，文件本体才会被回收

因此需要 inode 里有一个“还有多少名字指向它”的计数： `nlinks`

1.2 软链接 (symlink)

软链接做的事：创建一个新的 inode，但它里面存的是“目标路径字符串”，比如 `"../dir/real.txt"`。

结果：

- 软链接自己是一个独立对象
- 打开软链接时，系统会先读出它的字符串，再去找目标文件
- 如果目标被删了，软链接会“断掉”（因为它只是字符串）

2. 需要哪些数据结构改动

2.1 inode 增加链接计数

如果 SFS 的磁盘 inode 没有 `nlinks`，需要加：

```
#define SFS_TYPE_SYMLINK 4    // 示例：新增软链接类型

struct sfs_disk_inode {
    uint16_t type;           // 文件/目录/设备/软链接
    uint16_t nlinks;         // 硬链接计数（至少为 1）
    uint32_t size;           // 文件大小；软链接时为“路径字符串长度”
    uint32_t direct[NDIRECT];
    uint32_t indirect;
};
```

说明：

- 普通文件创建时： `nlinks=1`（至少有一个名字指向它）
- 每新增一个硬链接： `nlinks++`
- 每删除一个名字： `nlinks--`

2.2 软链接的存储方式

软链接 inode 的 `type = SFS_TYPE_SYMLINK`

它的内容就是目标路径字符串，存放在数据块中：

- 写入字符串时使用现有的“写文件”路径
- 读取字符串时使用现有的“读文件”路径

这样实现最直观，不需要再引入额外的存储结构。

3. 至少需要哪些接口（语义层面）

3.1 硬链接相关

- `int sys_link(const char *oldpath, const char *newpath);`
语义：在 `newpath` 的父目录里新增一个目录项，让它指向 `oldpath` 的 inode，并把 inode 的 `nlinks++`
- `int sys_unlink(const char *path);`
语义：删除目录项，让 inode 的 `nlinks--`；当 `nlinks==0` 且没人打开时回收 inode 和数据块
(可选但常见)
- `int sys_rename(const char *oldpath, const char *newpath);`
语义：把名字挪位置或改名，要求尽量“要么成功要么不改”（避免中途崩）

3.2 软链接相关

- `int sys_symlink(const char *target, const char *linkpath);`
语义：创建一个 `type=SYMLINK` 的 inode，在其中写入字符串 `target`，并在目录里放入 `linkpath -> inode`
- `ssize_t sys_readlink(const char *path, char *buf, size_t bufsz);`
语义：读取软链接 inode 的字符串（不去找目标文件）
- `int sys_lstat(const char *path, struct stat *st);`
语义：获取软链接自身信息（而不是目标文件信息）

4. 路径解析时如何处理软链接（避免死循环）

当系统解析路径 `a/b/c`，逐段查目录项得到 inode：

- 如果遇到 inode 类型是 SYMLINK：
 - `readlink/lstat`：不跟随，直接返回软链接自身
 - `open/stat`：需要跟随
做法是：读出它保存的目标字符串 `target`，把它当成新的路径继续解析

必须加一个“最大跟随次数”限制，避免 `x -> y`、`y -> x` 这种互相指导致无限循环：

- `follow_limit = 8`（或 40）
- 每跟随一次 `follow_limit--`
- 归零就报错 `-E_LOOP`

5. 同步互斥如何体现（避免并发导致 nlinks 与目录项不一致）

这里的核心问题很具体：

- `link` 不是只改一个地方：既要写目录项，又要改 inode 的 `nlinks`
- `unlink` 同理：既要删目录项，又要 `nlinks--`
- 如果两个线程同时做这些操作，很容易出现：
 - 目录项写了，但 `nlinks` 没加上

- `nlinks` 减成负数或少减/多减
- 同名目录项被重复创建

因此需要约束：**修改目录项和修改 `nlinks` 必须作为一个整体被保护起来。**

5.1 建议锁策略

- 父目录 inode 锁：保护该目录下目录项增删改
- 目标 inode 锁：保护 `nlinks` 修改与回收判定

建议固定加锁顺序，避免死锁：

1. 先锁父目录 inode
2. 再锁目标 inode

(如果涉及两个目录，比如 rename，从小 inode 号到大 inode 号依次加锁)

5.2 link 的一致性流程

`sys_link(old, new)`：

1. 找到 old 的 inode (需要存在)
2. 找到 new 的父目录 inode
3. 锁住 new 的父目录 inode
4. 再检查一次：new 是否已经存在 (避免并发下刚被创建)
5. 创建目录项 `newname -> old_inode`
6. 锁住 old inode
7. `old_inode->nlinks++`
8. 解锁并返回

如果第 5 步写目录项失败，就不能去第 7 步加 `nlinks`。

5.3 unlink 的一致性流程

`sys_unlink(path)`：

1. 找到父目录 inode + 目录项对应 inode
 2. 锁住父目录 inode
 3. 删除目录项
 4. 锁住目标 inode
 5. `nlinks--`
 6. 若 `nlinks==0`：
 - 若没人打开：释放 inode 和数据块
 - 若仍打开：仅标记“延迟删除”，等最后一个 close 才回收
 7. 解锁返回
-

6. 软链接创建与读取的关键点

- 创建 symlink 需要目录锁（避免同名）
- 写入目标字符串后基本不再改变
- readlink/lstat 只读数据，一般不需要复杂同步（最多 inode 级读锁）

本实验重要知识点与 OS 原理对照

本实验中的知识点	对应的 OS 原理知识 点	含义（实验视 角）	含义（原 理视角）	关系与差异理解
逻辑偏移到磁盘块的映射 (<code>sfs_bmap_load_nolock</code>)	文件系统的文件布局 与索引结构 (inode/ 索引块)	读文件时不能把 “文件第 blkno 块”当成“磁盘第 blkno 块”，必 须通过 inode 的映射找到真实 磁盘块号再读	文件由 inode 记 录元数据 与数据块 定位方式 (直接/间 接块 等)，通 过索引结 构实现“文 件视角的 连续字节 流”	原理强调“结构设计” (为何用索引、如 何扩展大文件)， 实验体现为“每次 I/O 前必须走 bmap”，差异在于 实验把复杂结构封 装在 bmap 内，使 用者只看到映射结 果
按块设备进行分层 I/O：部分 块读 (<code>sfs_rbuf</code>) 与整块读 (<code>sfs_rblock</code>)	I/O 子系统与块设备 抽象	读请求按字节给 出，但磁盘按块 访问，因此实现 要把请求拆成首 段/整块/尾段， 分别调用不同接 口	块设备提 供固定粒 度访问， 文件系统 在其之上 做缓存、 预读、合 并请求等 优化	实验的三段式拆分 是最直接的正确实 现；原理中还会进 一步讨论缓存一致 性、预读、调度 等，实验没有展开 性能策略但体现了 块粒度约束
EOF 与读范围裁剪 (<code>endpos</code> 裁剪到 <code>din->size</code>)	文件语义与系统调用 规范 (<code>read</code> 的返回 值)	读文件必须保 证：从 EOF 开 始读返回 0；读 超过文件大小只 返回可读部分	<code>read()</code> 以返回值 表示“实际 读取字节 数”，0 表 示 EOF	原理强调接口语 义；实验把语义落 实为具体边界条件 处理 (裁剪、提前 返回)，差异在于 实验需要把所有异 常路径明确写出来
VMA 建立 (<code>mm_map</code>)	虚拟内存管理：地址 空间、虚拟内存区域	装载 ELF 段前 先把虚拟区间登 记为“有效且具 有某些属性的区 域”，为后续页 表映射提供依据	进程地址 空间通常 由多个区 域组成， 每个区域 有权限与 用途 (代 码/数据/ 栈/堆/映 射文件)	原理强调“抽象与组 织方式”，实验体现 为“先登记后分配 页”，并把权限同时 体现在 VMA 与 PTE 上；实验中缺页按 需装载不完整，但 VMA 结构已具备扩 展空间

本实验中的知识点	对应的 OS 原理知识 点	含义（实验视 角）	含义（原 理视角）	关系与差异理解
页表建立与物理页分配 (<code>pgdir_alloc_page</code>)	页式存储管理、多级 页表、地址翻译	加载段时逐页分 配物理页并建立 映射，使用户虚 拟地址可访问	虚拟地址 通过页表 翻译到物 理页，权 限由页表 项控制	原理描述机制；实 验中直接“分配+填 充”，属于预加载策 略；差异在于现代 OS 常用按需分页 (page fault 时再分 配/读盘)，实验强 调可理解与可控的 装载流程
段权限从 ELF 标志映射到 PTE (R/W/X)	内存保护与访问控制	根据 <code>p_flags</code> 设置 <code>PTE_R/W/X</code> ， 避免代码段可写 等不合理权限	硬件通过 页表权限 位实现隔 离与最小 权限原则	实验体现“权限必须 从文件元信息传递 到硬件机制”；原理 还会讨论 <code>W^X</code> 、 ASLR 等策略，实验 只实现最核心的权 限对应关系
BSS 语义 (<code>p_memsz > p_filesz</code> 部分清零)	程序装载模 型: <code>.text/.data/.bss</code>	每页先清零，再 拷贝文件中存在 的部分，天然保 证 BSS 为 0	可执行文 件不存储 BSS 实际 内容，通 过装载时 分配并清 零实现	原理告诉“为什么 BSS 不占文件空 间”，实验用“先 <code>memset</code> 再 <code>copy</code> ”把 语义变成可验证的 实现；差异在于实 验按页处理，天然 处理了非页对齐情 况
用户栈建立与 <code>argc/argv</code> 构 造	进程启动过程与 ABI 约定	把字符串压入 栈，再构造 <code>argv[]</code> 指针数 组，并对齐 <code>sp</code>	用户态入 口约定参 数传递方 式与栈对 齐规则， 运行时依 赖这些假 设	原理通常从“调用约 定/运行时启动”讲概 念；实验必须处理 具体地址与对齐细 节，否则会出现“能 加载但一运行就异 常”的问题；差异在 于实验显式写出布 局与复制
切换页表与刷新 TLB (<code>lsatp / flush_tlb</code>)	上下文切换中的地址 空间切换、TLB 一致 性	新程序页表准备 好后必须让 CPU 使用新页 表，并清除旧翻 译缓存	TLB 缓存 地址翻 译，切换 页表时必 须保证不 会用旧缓 存导致越 权或错误 映射	原理强调“为什么需 要 flush”；实验把它 放在装载完成后、 返回用户态前，这 是保证取指与访存 正确的关键步骤
trapframe 设置 (<code>epc/sp/a0/a1/status</code>)	用户态/内核态切 换、异常返回机制 (<code>trap/sret</code>)	通过设置 trapframe 决定 <code>sret</code> 返回后 CPU 的 PC、 SP、参数寄存 器与特权级	处理器通 过保存/恢 复寄存器 现场完成 特权级切 换，返回 地址由异 常返回寄 存器决定	原理给出机制；实 验体现“启动新程 序就是构造一份正 确的返回现场”，差 异在于实验把启动路 径压缩到可观测的 字段设置上

本实验中的知识点	对应的 OS 原理知识点	含义（实验视角）	含义（原理视角）	关系与差异理解
文件系统与执行的衔接（fd 读取 ELF）	统一资源抽象：文件即字节流、exec 装载器	exec 不再依赖内存镜像，而是从文件系统读取可执行文件内容	UNIX 把可执行文件当作普通文件，通过 VFS/FS 提供统一访问	原理强调“统一接口”；实验体现“load_icode_read 只需要 fd”，将执行与存储通过文件抽象连接起来

OS 原理中很重要但本实验未直接覆盖的知识点

- 虚拟内存的按需分页（Demand Paging）与懒加载（Lazy Loading）
本实验采用“预加载”：装载时把段内容一次性读入并分配页；而现代 OS 常在缺页异常时按需分配并从磁盘读入。
- 页面置换与回收策略（LRU/Clock、工作集等）
实验涉及分配页与映射，但不涉及内存压力下如何选择淘汰页、如何与交换区协作。
- 页缓存（Page Cache）与缓冲缓存（Buffer Cache）的统一与一致性
实验中读块接口可用，但未体现真实 OS 中对文件 I/O 的缓存、回写、脏页管理与一致性维护。
- 进程间通信（IPC）的系统化机制
本实验只实现“执行”，不涉及管道、信号、共享内存、消息队列等 IPC 机制的系统集成与语义细节。
- 安全机制：权限模型、用户身份、文件访问控制（UID/GID/ACL）
实验中执行只基于文件内容是否为 ELF，并未实现真实 OS 的权限检查（可执行位、所有者、访问控制等）。
- 可执行文件的动态链接与运行时装载（ELF 动态段、解释器、重定位）
实验以静态或简化 ELF 为主，不涉及动态链接器、GOT/PLT、重定位处理等。
- 多核下的同步与内存一致性（锁、原子指令、屏障）
实验主要在单核教学场景下实现，未覆盖多核并发下页表修改、TLB shutdown、文件系统锁的复杂性。